Campainha com registro fotográfico e notificações

Disciplina: INE5670 - Desenvolvimento de Sistemas Móveis e Embarcados, 2026.2.

Alunos: Gabriela da Silva de Jesus e Eduardo de Oliveira Chaves.

Descrição do sistema

O projeto propõe uma campainha que registra uma foto quando o visitante pressiona um botão. Um buzzer fornece o aviso sonoro local. O Raspberry Pi associa a imagem à data e ao horário do acionamento e mantém o registro no cartão microSD. Na integração prevista, um backend receberá os eventos e permitirá que o morador consulte as fotos no aplicativo móvel e receba notificações de novas visitas.

1. Problema e objetivo

Uma campainha convencional avisa sobre a presença de alguém, mas não mantém uma imagem que o morador possa consultar depois. O objetivo é reunir aviso sonoro, registro fotográfico e consulta pelo celular, inclusive quando o morador não puder atender no momento do acionamento.

Funcionalidades do escopo principal

• Registrar cada acionamento do botão e acionar o buzzer.

• Capturar uma foto com a câmera CSI e associá-la ao evento.

• Armazenar imagem, horário e estado de processamento localmente.

• Disponibilizar os registros no backend na etapa de integração.

• Permitir consulta posterior de fotos e histórico no aplicativo móvel.

• Notificar o morador sobre novas visitas pelo aplicativo.

2. Funcionamento do sistema proposto

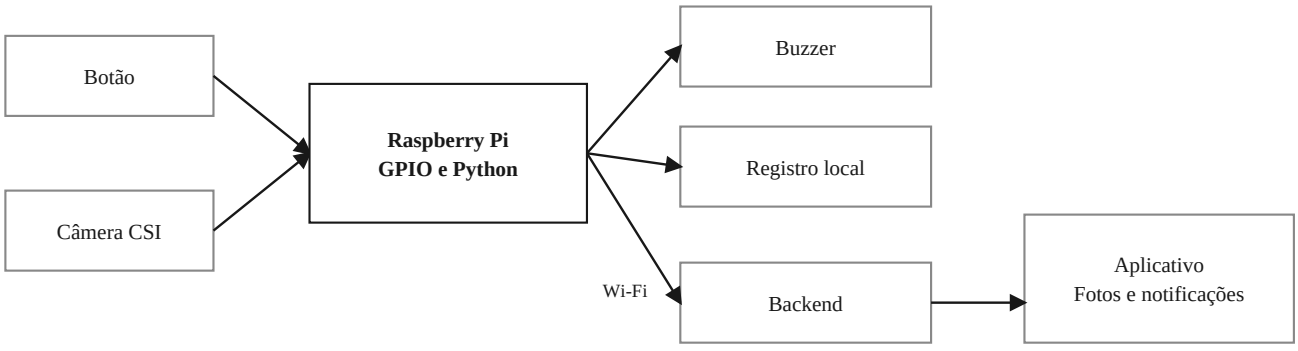


Figura 1 - Arquitetura prevista para integrar o embarcado, o backend e o aplicativo.

O acionamento cria um evento na fila local. O processamento aciona o aviso sonoro, solicita a fotografia e atualiza o registro. Na etapa de integração, o backend receberá a imagem e os metadados, confirmará o evento e disponibilizará a consulta e a notificação ao usuário.

3. Esquema eletrônico e plataforma embarcada

Componentes e ligações

A montagem principal utiliza Raspberry Pi 3 Model B+, botão momentâneo, buzzer, câmera CSI OV5647, protoboard, jumpers, cartão microSD e fonte. A câmera será conectada pela interface CSI; o botão e o comando do buzzer usam o conector GPIO.

Componente	Conexão prevista	Pino físico
Botão, terminal 1	GPIO17 com pull-up interno	11
Botão, terminal 2	GND	9
Comando do buzzer	GPIO18, PWM; interface de acionamento conforme o componente	12
Referência do buzzer	GND comum com a interface de acionamento	14
Câmera CSI OV5647	Cabo flat no conector CSI	Conector dedicado
Alimentação da placa	Fonte micro-USB de 5 V adequada ao Raspberry Pi 3 B+	Entrada de alimentação

A biblioteca GPIO Zero utiliza a numeração BCM. Assim, `Button(17)` se refere ao GPIO17, localizado no pino físico 11. O botão fecha a entrada ao GND quando pressionado; o pull-up interno mantém o estado de repouso.

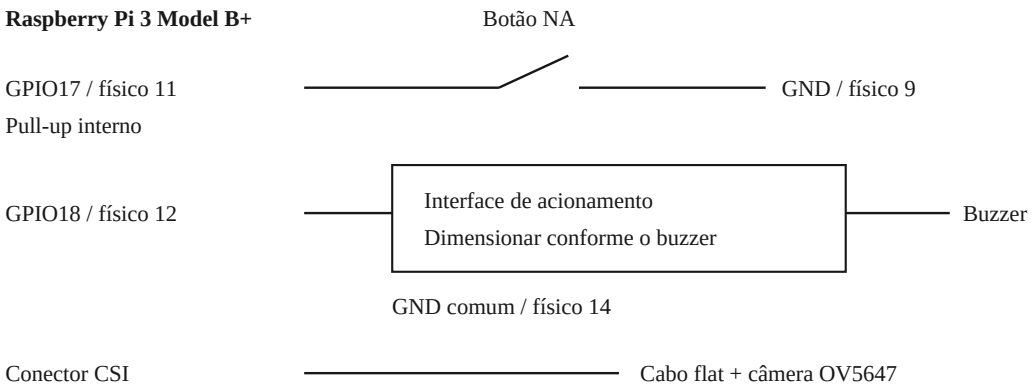


Figura 2 - Ligações funcionais do botão, da câmera e do comando sonoro. O estágio de acionamento do buzzer será dimensionado conforme suas características elétricas.

Os GPIOs trabalham com sinais de 3,3 V. O modelo, a tensão e a corrente nominal do buzzer devem orientar a necessidade de transistor, resistor e proteção no acionamento. O sinal PWM define o comando sonoro; não substitui esse dimensionamento. A alimentação do buzzer e sua interface devem compartilhar a referência de terra da placa.

Plataforma

Será utilizado Raspberry Pi OS, com Python para o programa embarcado, GPIO Zero para botão e buzzer e `rpicam-still` para fotografias. O código admite `libcamera-still` como alternativa em instalações compatíveis. SQLite mantém os eventos no microSD. Wi-Fi será o meio de comunicação com o backend.

O Raspberry Pi 3 Model B+ é um computador de placa única com 1 GB de memória, conector CSI e GPIO de 40 pinos. A execução em Linux permite reunir acesso aos periféricos, armazenamento local e comunicação de rede na mesma plataforma.

Código comentado e organização

O programa principal é `main.py`. O módulo `campainha.py` concentra a fila e a persistência dos eventos. O acionamento do botão registra o evento rapidamente; uma thread realiza as capturas em sequência, para que a execução da câmera não ocupe o callback do botão.

O trecho abaixo reproduz a configuração do botão e do buzzer e a operação de captura usada pelo programa:

```
# GPIO Zero usa numeração BCM; pull-up mantém o botão solto em nível alto.
botao = recursos.enter_context(
    Button(17, pull_up=True, bounce_time=0.05))
buzzer = recursos.enter_context(
    PWMOutputDevice(18, frequency=2000, initial_value=0))

# A função é chamada pela thread de captura, após retirar um evento da fila.
def capturar(arquivo):
    buzzer.value = 0.5
    parar.wait(0.2)
    buzzer.off()
    subprocess.run(
        [camera, "--nopreview", "--timeout", "1000",
         "--output", str(arquivo)],
        check=True, timeout=30, capture_output=True, text=True,
    )
```

O código completo, com tratamento das falhas e encerramento dos recursos, acompanha a entrega e está no GitHub do projeto.

Arquivo	Responsabilidade
main.py	Botão, buzzer, câmera e ciclo de execução embarcado
campainha.py	Eventos, fila de captura, SQLite e retenção local
run.sh	Iniciar o programa junto com o facilitador de envio ao computador
enviar_fotos.py	Transferir fotos pendentes ao receptor de bancada
scripts/	Preparar a conexão e receber os arquivos no computador

Armazenamento e tratamento de falhas

Cada evento recebe um identificador persistente no banco. As fotos usam nomes no formato `HH-MM-SS__Dia-Mes.jpg`, como `19-41-23__25-10.jpg`; a data completa permanece nos metadados. A captura aguarda outro segundo caso o nome já exista, preservando a imagem anterior.

O programa registra falhas de captura e mantém o limite padrão de 200 imagens. A retenção remove somente fotos marcadas como enviadas. Se todas estiverem pendentes, novos eventos podem ficar sem imagem, com o erro registrado. O histórico de metadados ainda exigirá uma política de retenção para uso prolongado.

4. Integração com backend e aplicativo móvel

Consulta posterior de fotos

O backend deverá receber os metadados e a imagem de cada visita, manter o histórico e fornecer uma interface de consulta ao aplicativo. O identificador persistente do evento permitirá reconhecer um envio repetido e evitar duplicatas. A confirmação de recebimento deverá indicar o mesmo identificador enviado pelo dispositivo.

O aplicativo terá uma lista de visitas com data e horário. Ao selecionar um registro, o usuário poderá visualizar a foto correspondente ou a indicação de falha de captura. O acesso deverá ser associado ao usuário e ao dispositivo autorizado.

Notificações de novas visitas

Após receber um novo evento, o backend deverá solicitar uma notificação ao celular do morador. O aplicativo permitirá abrir o registro relacionado ao aviso e consultar a imagem. A entrega da notificação dependerá da rede, das permissões do aplicativo e do serviço de notificações escolhido.

Backend, aplicativo e integração das notificações serão desenvolvidos nas etapas seguintes. As tecnologias serão definidas conforme as atividades da disciplina. A proposta não assume um serviço específico de notificações nem apresenta essa integração como concluída.

Execução local e apoio à demonstração

Executar `python3 main.py` permite operar a parte embarcada e salvar registros localmente. O facilitador `run.sh` acrescenta um processo separado para enviar as fotos ao computador; esse apoio serve à preparação e à demonstração da montagem.

O computador inicia a conexão SSH com o Raspberry e disponibiliza um receptor restrito ao localhost por um túnel. A autenticação SSH utiliza chave, sem instalar a chave privada no Raspberry e sem habilitar acesso SSH ao computador. O facilitador exige que o recebimento esteja disponível antes de iniciar e limita o envio a três tentativas por foto. O contador persiste após reinício; ao esgotar o limite, o arquivo permanece na placa para recuperação manual. Outros arquivos continuam elegíveis para envio.

O receptor de bancada não oferece histórico ao celular nem dispara notificações. Essas responsabilidades pertencem ao backend e ao aplicativo previstos para o projeto integrador.

Melhorias possíveis até a entrega final

Sensor PIR para registrar movimento e LED para indicar estado poderão ser avaliados como melhorias, conforme disponibilidade e tempo de integração. Eles não compõem o escopo principal desta proposta. Reconhecimento facial e vídeo contínuo também não integram a solução proposta.

As imagens da demonstração serão registradas com participantes cientes da captura. Credenciais e fotos de visitantes permanecerão fora do repositório público e do pacote de fontes.

5. Estimativa de custos do protótipo

O orçamento considera o desembolso adicional para montar o núcleo proposto com os materiais já disponíveis. Reutilização não significa que os equipamentos tenham valor de mercado nulo; significa que sua aquisição não é uma nova despesa do projeto.

Item	Situação prevista	Nova aquisição (R$)
Raspberry Pi 3 Model B+	Reutilização	0,00
Câmera CSI OV5647 e cabo	Reutilização	0,00
Botão momentâneo e buzzer	Reutilização	0,00
Protoboard e jumpers	Reutilização	0,00
Fonte e cartão microSD	Reutilização	0,00
Computador e celular	Uso dos equipamentos disponíveis	0,00
Subtotal de novas aquisições dos itens disponíveis		0,00
Componentes de acionamento e proteção do buzzer	Dimensionar e cotar conforme o componente	A definir

O custo adicional final permanece aberto para eventuais ajustes de acionamento, conexões e acondicionamento. PIR e LED serão orçados apenas se incorporados como melhorias. Hospedagem, frete e substituição de equipamentos não estão incluídos. Esta estimativa não apresenta os preços antigos de reposição como novas compras.

6. Cronograma do projeto

Ordem	Etapa	Entrega prevista
1	Definição do escopo	Funcionalidades e arquitetura do sistema
2	Preparação da placa	Sistema operacional, Wi-Fi e ambiente Python
3	Montagem embarcada	Botão, câmera e acionamento do buzzer
4	Registro local	Fila, fotografias, metadados e tratamento de falhas
5	Backend	Recebimento, armazenamento e consulta dos eventos
6	Aplicativo móvel	Histórico, visualização de fotos e notificações
7	Integração e demonstração	Fluxo completo entre embarcado, backend e celular
8	Entrega final	Documentação e fontes alinhados; melhorias opcionais avaliadas

Referências técnicas

Raspberry Pi 3 Model B+; documentação de hardware e GPIO; GPIO Zero: dispositivos de saída; software de câmera rpicam-apps; repositório do projeto.